

WPF의 현재와 미래

.NET10 시대에서 다시 바라본 데스크톱 개발

박송화
2025.11.27

Contents

01 자기 소개

02 역사 및 탄생 배경

03 활용 영역

04 강점 및 한계점

05 지원종단

06 .NET 10 in WPF

07 전망

박송화 / Software Developer

✓ 경력

- 크린팩토메이션(CFI) Soft팀
- 장비 프로그램 10년차 개발자
- SK Hynix, 삼성전자 반도체 장비 프로그램, 모니터링 프로그램 개발

✓ 블로그 및 연락처

- <https://westahn.com/> 블로그 운영 중
- westahn.deve@gmail.com



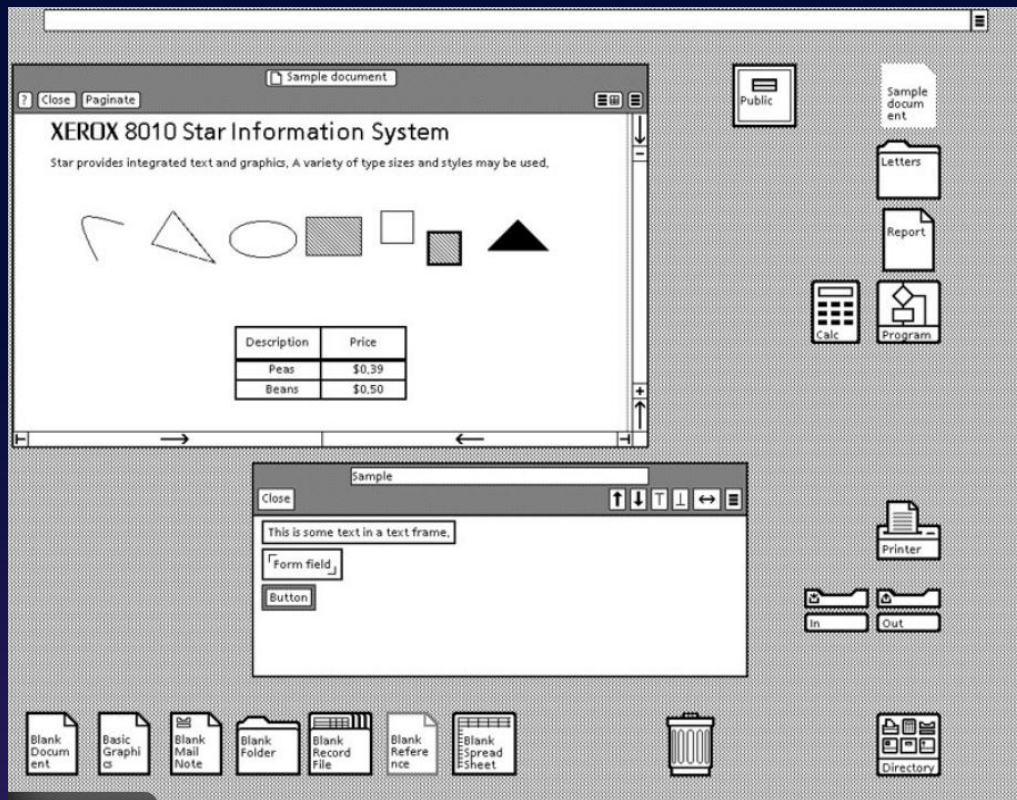
1980s - GUI의 탄생

Xerox PARC GUI 탄생

인간과 컴퓨터 간 상호작용을 위해 Graphic User Interface 탄생

텍스트 명령어 기반(CLI)에서 WIMP도입

Windows, Icon, Menu, Pointer 개념을 도입하여 직관적이고 사용하기 쉬운 환경을 제공



Xerox 8010 Star Information System

2 역사

.NET Conf 2025 @Seoul

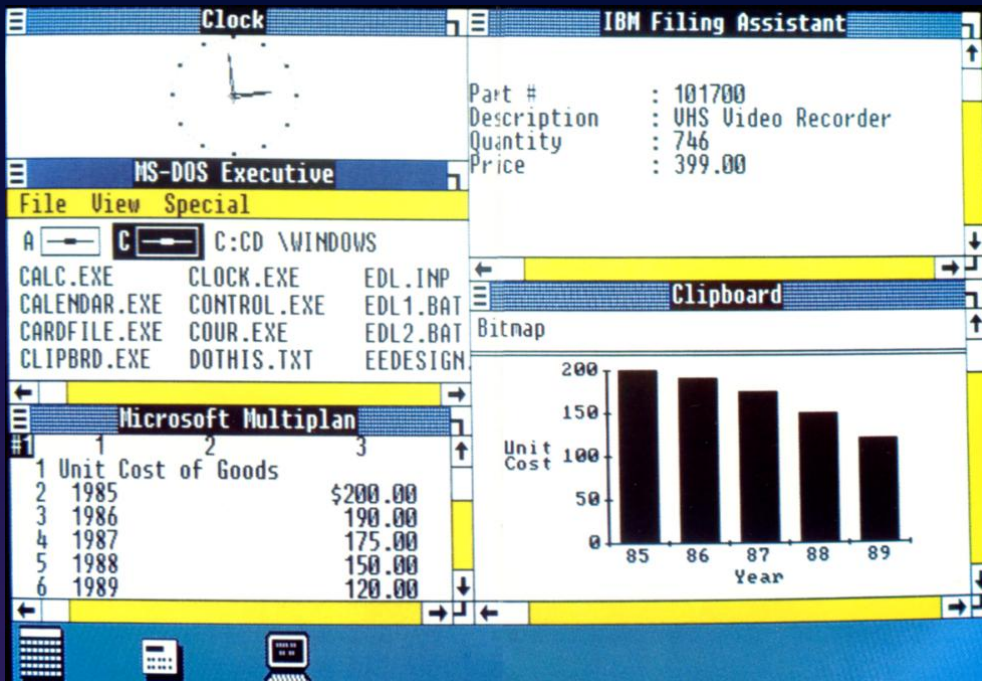
1985s - Windows 1.0

Microsoft 첫 GUI

- MS-DOS 위에서 동작하는 16비트 환경
- 마우스 지원과 기본적인 내장 애플리케이션을 포함

GUI 대중화의 초석

- 타일 방식으로 정렬(창이 겹쳐지지 않음)
- 하나 프로그램 다운시 전체에 영향 끼침



2 역사

.NET Conf 2025 @Seoul

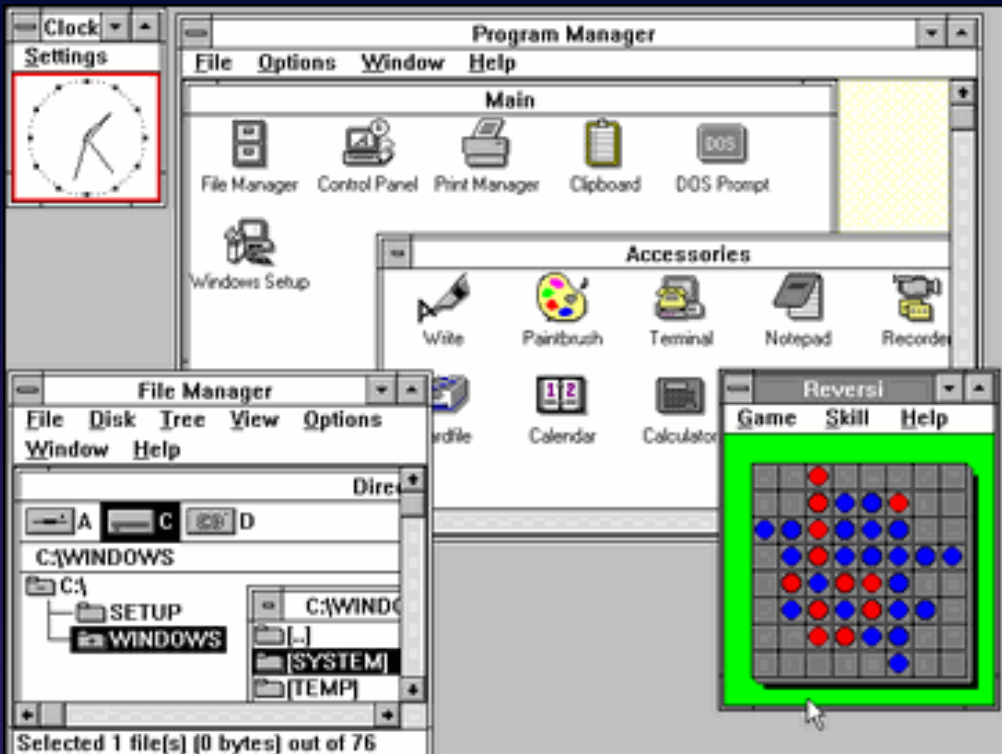
1990s - Windows 3.0

PC GUI 대중화 기여

- 향상된 가상 메모리 지원, 보호 모드 실행, 멀티 태스킹, 글꼴 지원으로 대중화
- 사용자 친화적인 인터페이스와 안전성, 네트워크 기능 강화

새로운 GUI 체계 도입

- 프로그램 관리자(Program Manager)와 파일 관리자(File Manager)라는 새로운 GUI 체계 도입
- 드래그 앤 드롭 기능 도입



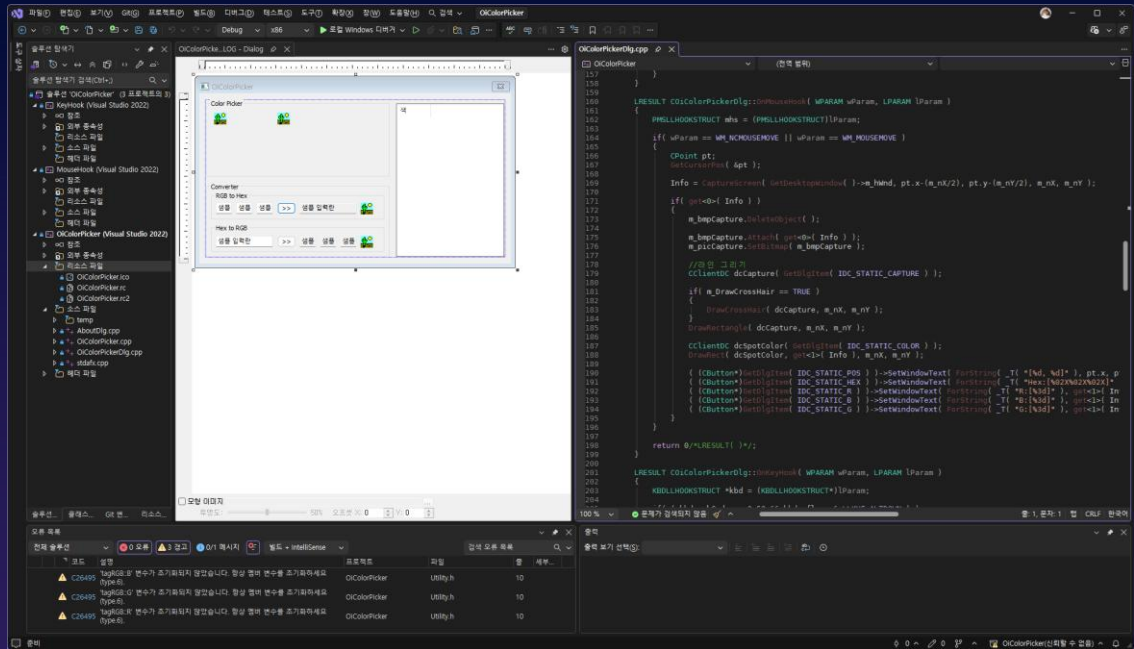
2 역사

.NET Conf 2025 @Seoul

1992 - MFC등장

MFC등장

- MFC(Microsoft Foundation Class Library)
- Microsoft에서 Windows 개발자들이 C++ 기반으로 효율적으로 데스크탑 프로그램을 만들 수 있도록 발표한 객체지향 라이브러리
- 1990년대~2000년대 다양한 산업용 소프트웨어 제작에 활용되었음



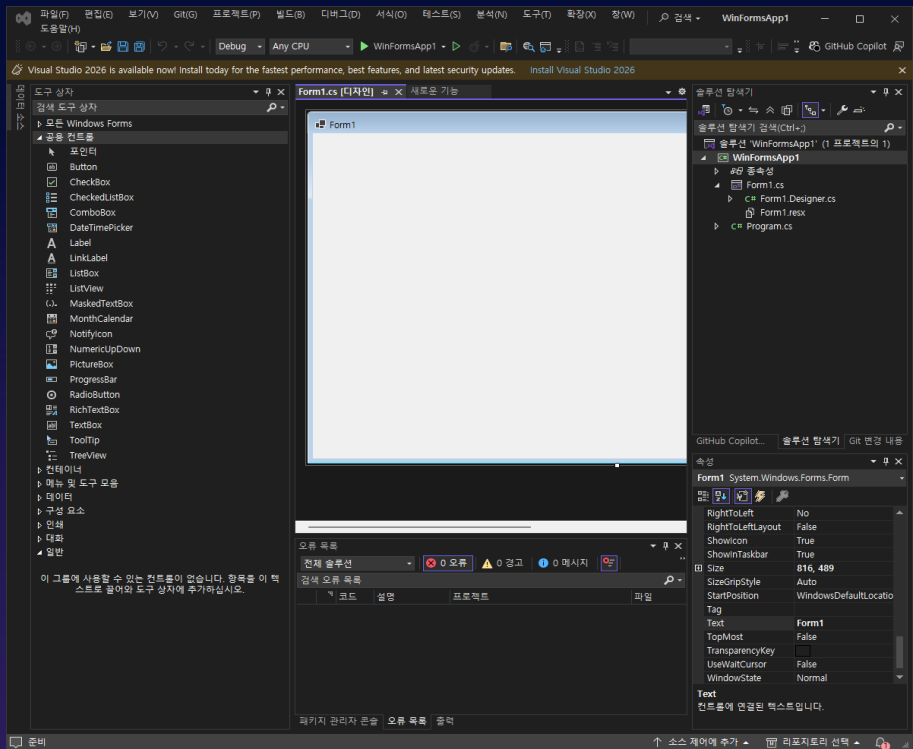
2 역사

.NET Conf 2025 @Seoul

2002 - Winform

Winform등장

- .NET Framework 1.0 출시에 맞춰 공개된 Microsoft 첫 번째 .NET 기반 데스크톱 UI 프레임 워크
- 기존 MFC 및 Win32 API 환경은 C/C++ 위주로 복잡하고 생산성이 낮았음
- 자동 메모리 관리(GC), 드래그 앤 드롭 디자인, 이벤트 기반 모델, GDI/GDI+ 기반 컨트롤



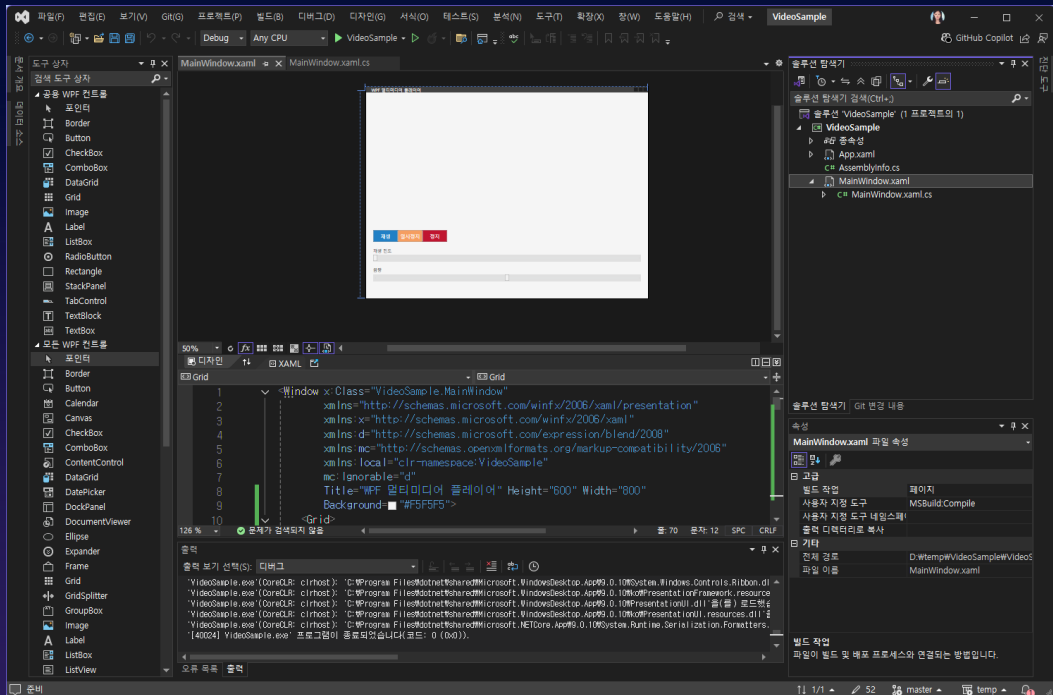
2 역사

.NET Conf 2025 @Seoul

2006 - WPF

WPF등장

- Windows Presentation Foundation
- .NET Framework 3.0과 함께 발표된 차세대 Windows 데스크탑 UI
- XAML(Extensible Application Markup Language)로 디자인하고 CodeBehind에서 코드를 작성
- DirectX 기반 벡터 렌더링 지원
- 현대적인 UI/UX, 미디어, 2D/3D 그래픽, 데이터 바인딩, 스타일링지원



WPF가 탄생하게 된 역사들

WPF(Windows Presentation Foundation)는 Microsoft의 진화 과정에서
Windows 애플리케이션 개발의 한계를 극복하고, 더욱 현대적이고 강력한 UI를 구축하기 위해 탄생했습니다



WPF 이후의 Framework

a Cross-platform XAML / C#
application framework

Xamarin
(2011~2024)

Xaml 기반의 크로스 플랫폼
UI 프레임워크

Avalonia
(2013~Present)

Windows UI Library
Windows 앱 개발용 UI 프레임워크

WinUI
(2018~Present)

.NET Multi-platform App UI
오픈 소스 크로스 플랫폼

MAUI
(2022~Present)

WinRT
(2012~2015)

Windows Runtime
Windows 8부터 도입된 Application

UWP
(2015~2020)

Universal Windows Platform
Windows 10부터 도입된 Application

Uno
(2018~Present)

Uno Platform
.NET 기반의 오픈소스 크로스
플랫폼 UI 프레임 워크

Avalonia XPF
(2023~Present)

WPF 애플리케이션을
크로스플랫폼에서 실행할 수 있는
상업용 솔루션

3 활용 영역

.NET Conf 2025 @Seoul



제조 및 장비 산업

장비 상태, 제어, 공정 모니터링 등 실시간
데이터 표시 및 직관적인 UI



의료 및 헬스케어

대용량 의료 이미지나 3D데이터 로드, 정밀한
2D/3D 그래픽 환경, 각 의료장비에 맞는
시뮬레이션 제공



산업용 솔루션

대규모 ERP, CRM, 업무 관리용 데스크톱
어플리케이션 개발에 사용
방대한 데이터 처리, 기업에 맞는 커스터
마이징

Software type by primary programming language

Python	Java	JavaScript	TypeScript	SQL	C#	C++	PHP	Go	Kotlin	
28%	31%	75%	71%	42%	38%	6%	77%	25%	15%	Websites
25%	35%	20%	23%	34%	39%	5%	41%	43%	24%	Web services
25%	25%	11%	16%	20%	28%	11%	16%	39%	20%	Cloud services
23%	35%	28%	33%	44%	46%	16%	34%	21%	37%	Business applications
20%	5%	4%	5%	13%	7%	6%	7%	4%	4%	Analytical software
17%	22%	10%	9%	17%	23%	34%	14%	19%	19%	System software
15%	12%	5%	5%	8%	10%	10%	7%	20%	7%	IT infrastructure
11%	19%	10%	11%	21%	16%	10%	9%	11%	17%	Finance
8%	7%	6%	6%	5%	8%	15%	5%	8%	8%	Development tools
7%	5%	3%	2%	3%	5%	9%	3%	7%	6%	Security
6%	5%	6%	6%	3%	7%	11%	5%	5%	15%	Entertainment
4%	1%	1%	1%	1%	2%	5%	1%	1%	2%	Home automation
4%	6%	8%	6%	4%	4%	3%	4%	4%	9%	Mini-apps
4%	4%	5%	3%	2%	14%	20%	2%	3%	7%	Games
3%	2%	1%	0%	1%	2%	17%	0%	3%	2%	Hardware
1%	1%	1%	0%	0%	4%	5%	1%	0%	1%	Augmented reality / Virtual reality

0%

77%

4 강점과 한계점

.NET Conf 2025 @Seoul

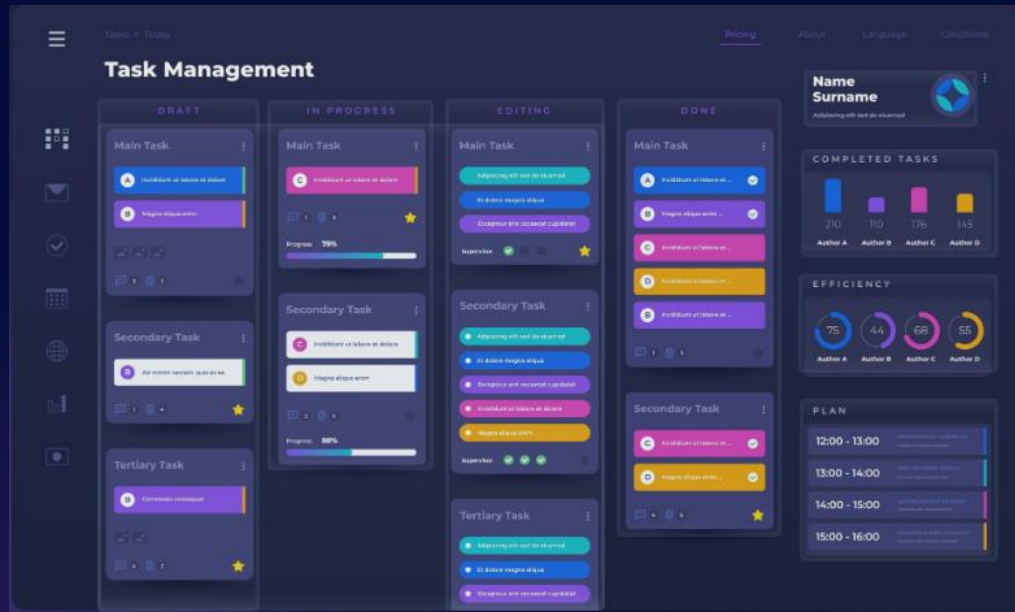
WPF 강점 01

유연한 스타일 및 템플릿

복잡한 UI 요구에 대응

커스터마이징 자유도

일관성 있는 UI적용



4 강점과 한계점

.NET Conf 2025 @Seoul

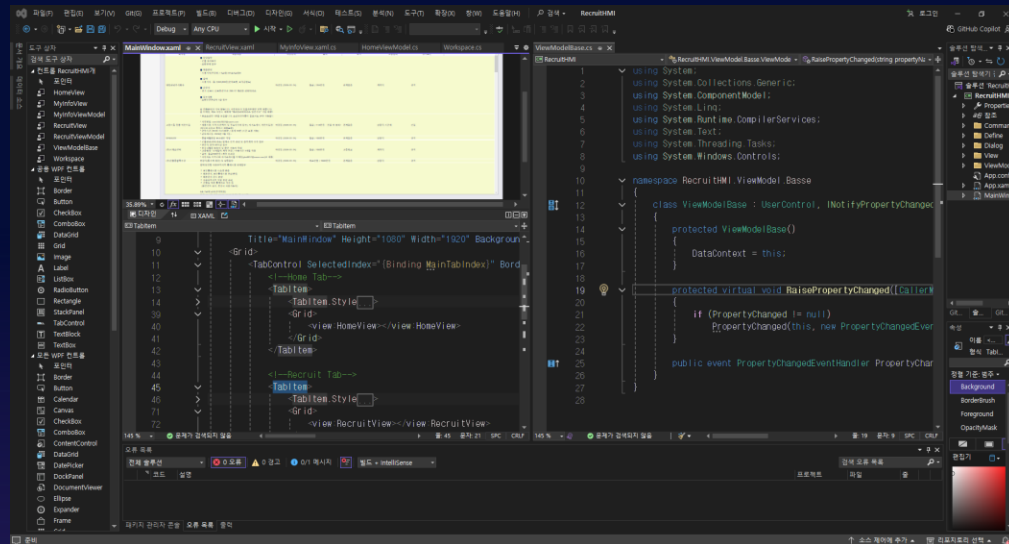
WPF 강점 02

XAML 기반 UI 설계

UI와 동작 코드 분리(MVVM패턴)

생산성과 가독성

재사용성과 확장성



4 강점과 한계점

.NET Conf 2025 @Seoul

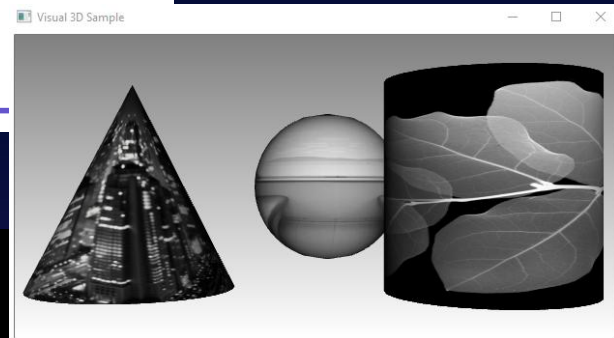
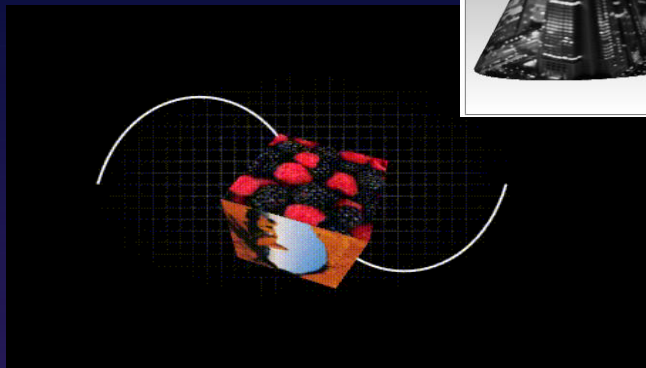
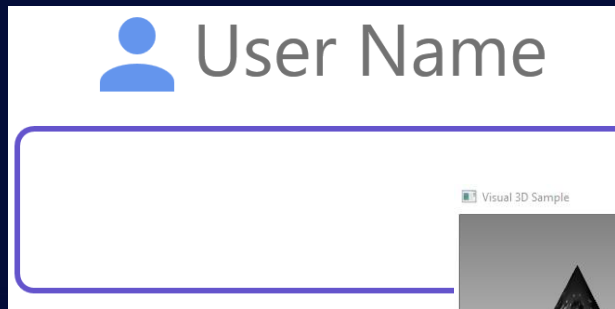
WPF 강점 03

고급 그래픽 및 애니메이션 지원

벡터 기반으로 깨지지 않음

DirectX 엔진 사용으로 2D, 3D 그래픽,
애니메이션 사용

오디오, 비디오등 미디어 요소와 결합



4 강점과 한계점

.NET Conf 2025 @Seoul

WPF 강점 04

안정적인 개발환경

산업현장에서의 검증

풍부한 서드파티

장기적 확장성

상용 UI 라이브러리



무료, 오픈소스 UI 라이브러리

MahApps, Metro, Material Design in XAML Toolkit

MVVM, 인프라 라이브러리

Prism, Caliburn.micro, ReactiveUI

4 강점과 한계점

.NET Conf 2025 @Seoul

WPF 한계점 01

크로스 플랫폼 미지원

운영체제 종속성

마이그레이션 어려움

서드파티 호환 제한



4 강점과 한계점

.NET Conf 2025 @Seoul

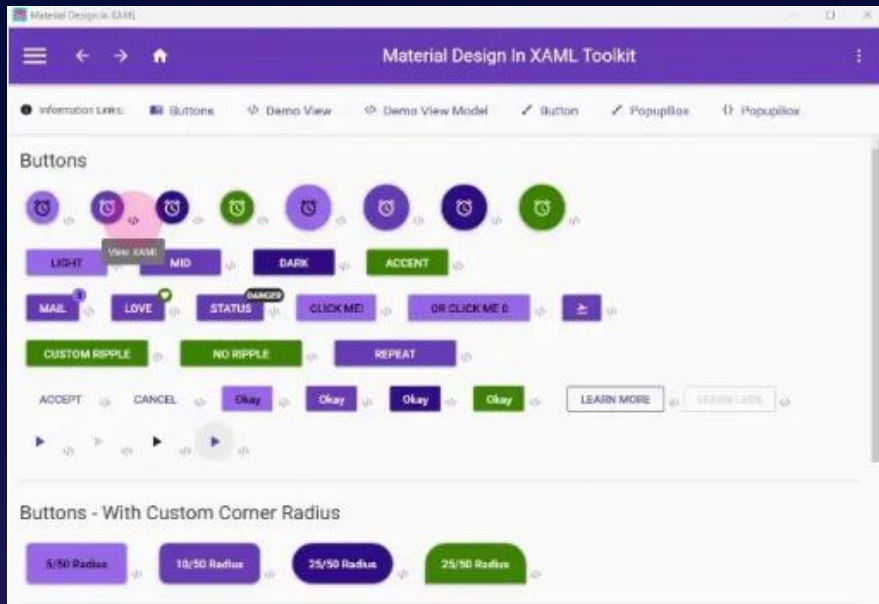
WPF 한계점 02

디자인 어려움과 그래픽 한계

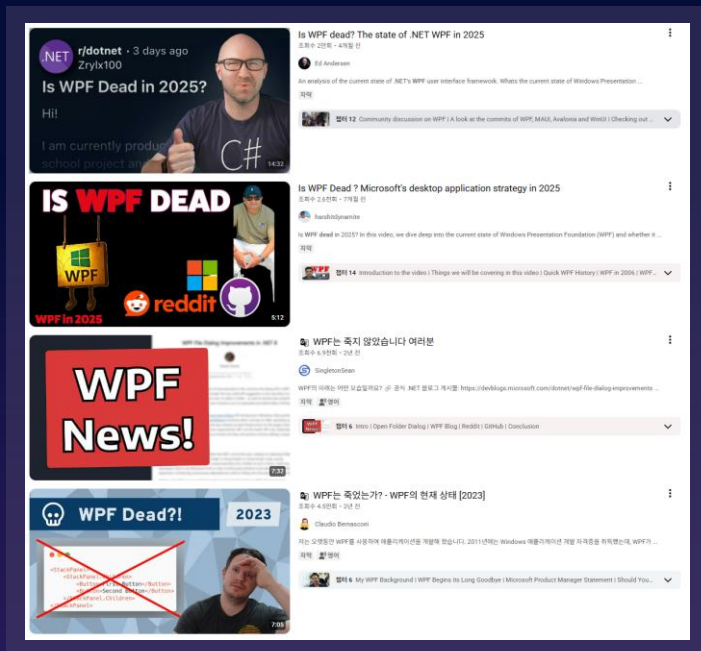
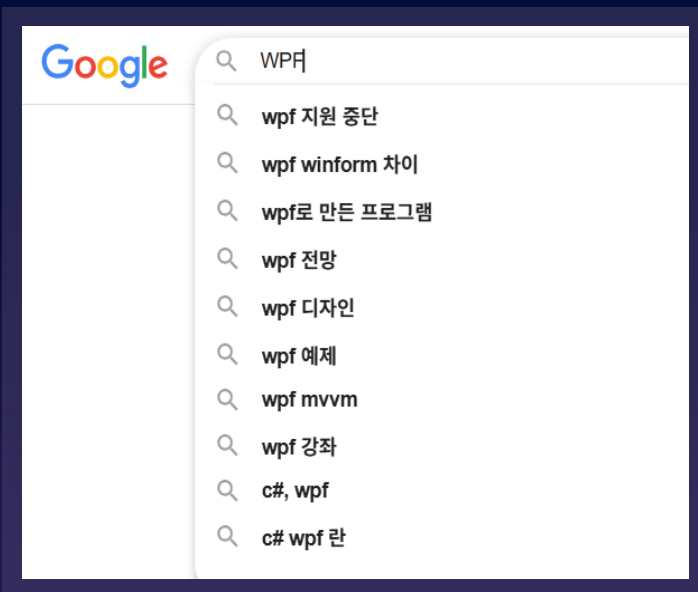
개발자의 디자인 강점 살리기 어려움

XAML과 MVVM패턴 이해 어려움

그래픽 및 애니메이션 메모리 제한

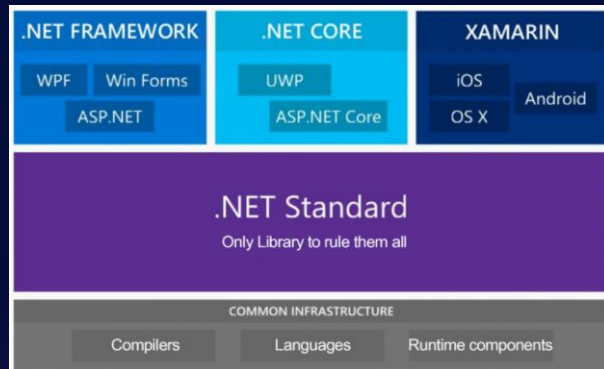
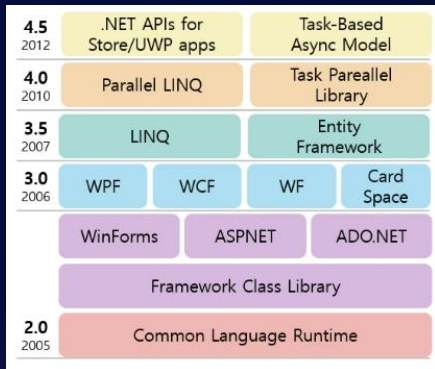


WPF는 지원이 중단된 것인가?



5 지원 중단

.NET Conf 2025 @Seoul

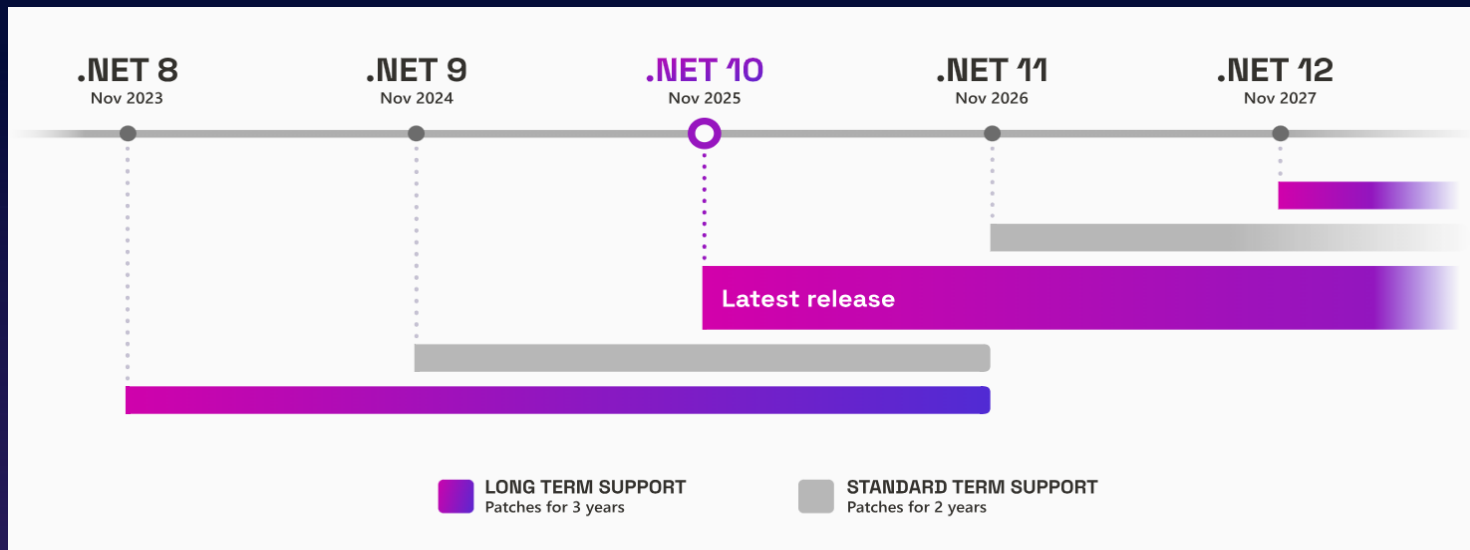


버전	플랫폼	표준	출시일	Visual Studio 대응 버전	변경 사항
1.0	.NET Framework 1.0		2002년 1월	.NET 2002	링크
1.1	.NET Framework 1.1	ECMA-334	2003년 4월	.NET 2003	-
2.0	.NET Framework 2.0		2005년 11월	2005	링크 ^[5]
3.0	.NET Framework 2.0~3.5	-	2007년 11월	2008	링크 ^[6]
4.0	.NET Framework 4.0	-	2010년 4월	2010	링크 ^[7]
5.0	.NET Framework 4.5	ECMA-334	2012년 8월	2012	링크 ^[8]
6.0	.NET Framework 4.6 .NET Core 1.0 .NET Core 1.1	-	2015년 7월	2015	링크 ^[9]
7.0	.NET Framework 4.6.2	-	2017년 3월	2017 v15.0	링크 ^[10]
7.1	.NET Framework 4.7 .NET Core 2.0	-	2017년 8월	2017 v15.3	링크 ^[11]
7.2	.NET Framework 4.7.1	-	2017년 11월	2017 v15.5	링크 ^[12]
7.3	.NET Framework 4.7.2 .NET Core 2.1 .NET Core 2.2	-	2018년 5월	2017 v15.7	링크 ^[13]
8.0	.NET Framework 4.8 .NET Core 3.0 .NET Core 3.1	-	2019년 9월	2019 v16.3	링크 ^[14]
9.0	.NET 5.0	-	2020년 11월	2019 v16.8	링크 ^[15]
10.0	.NET 6	-	2021년 11월	2022	링크
11.0	.NET 7	-	2022년 11월	2022 v17.4	링크 ^[16]
12.0	.NET 8	-	2023년 11월	2022 v17.8	링크 ^[17]
13.0	.NET 9	-	2024년 11월	2022	링크 ^[18]
14.0	.NET 10	-	2025년 11월	2026	링크 ^[19]

6 .NET10 in WPF

.NET Conf 2025 @Seoul

버전	원래 릴리스 날짜	최신 패치 버전	패치 릴리스 날짜	릴리스 유형	지원 단계	지원 종료
.NET 10	2025년 11월 11일	10.0.0	2025년 11월 11일	LTS	활성	2028년 11월 14일
.NET 9	2024년 11월 12일	9.0.11	2025년 11월 11일	STS	활성	2026년 11월 10일
.NET 8	2023년 11월 14일	8.0.22	2025년 11월 11일	LTS	활성	2026년 11월 10일



6 .NET10 in WPF

.NET Conf 2025 @Seoul

성능 향상

내부 데이터 구조 최적화

디스크/내부 구조를 .NET의 기본 타입(ReadOnlyCollection<T>등)으로 교체하여 메모리 할당을 줄임

- ReadOnlyCollection이란?
 - “읽기 전용 컬렉션”으로, 컬렉션 내용을 바꿀 수 없게 만드는 래퍼(wrapper)클래스
 - 불필요한 데이터 복사 및 중복 메모리 할당 없이 읽기 전용으로 제공되어 메모리 할당량이 줄어듦

글꼴 및 리소스 처리 개선

글꼴(Font) 컬렉션 로더를 관리(Managed) 코드로 마이그레이션하고, 캐시 작업을 최적화하여 텍스트 렌더링 속도 향상

XAML 파싱 속도

XAML을 읽어들이는(Parsing)속도와 정규식(Regex)사용이 최적화 되었음

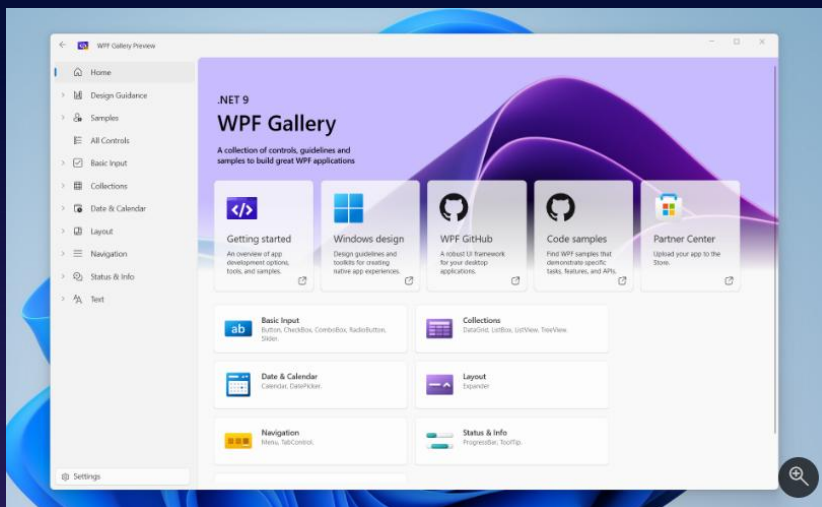
6 .NET10 in WPF

.NET Conf 2025 @Seoul

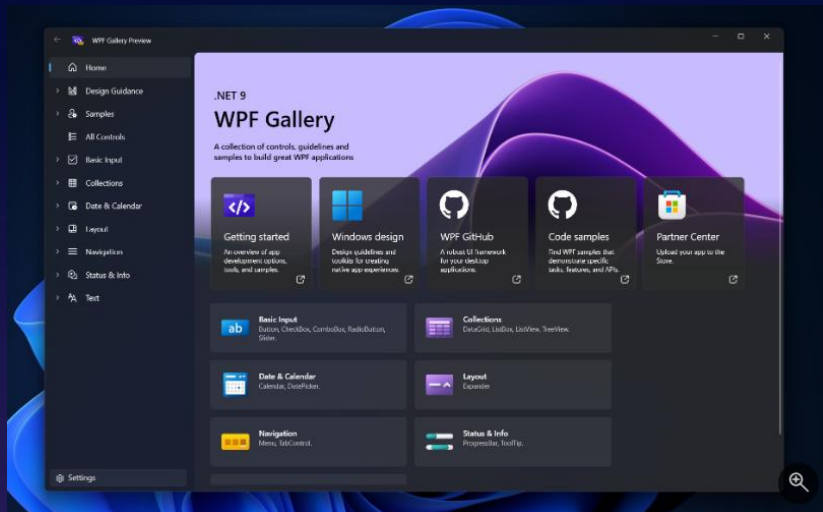
Fluent UI 스타일 확대

Fluent UI 스타일이란?

"빛, 깊이, 움직임, 재질, 스케일"이라는 5가지 요소를 통해 평면적인 WPF 앱을 입체적이고 감각적인 Windows 네이티브 앱처럼 변모시키는 디자인 시스템



Light Mode



Dark Mode

6 .NET10 in WPF

.NET Conf 2025 @Seoul

Fluent UI 스타일 적용법

Phase1 : App.xaml

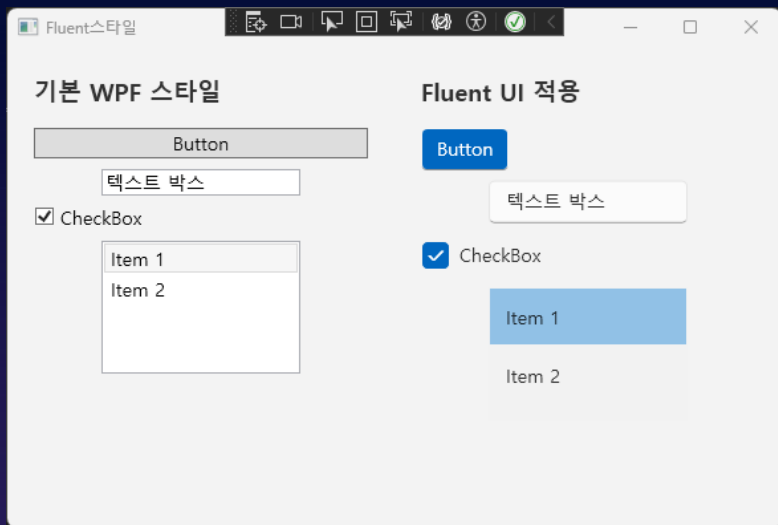
리소스사전 테마 적용

Phase2 : ThemeMode

Light, Dark, System, None 적용

Phase3 : Fluent UI 적용

테마 컬러 직접 적용 가능



```
<Application.Resources>
  <ResourceDictionary>
    <ResourceDictionary.MergedDictionaries>
      <!-- .NET 10 Fluent Theme -->
      <ResourceDictionary Source="pack://application:,,,/PresentationFramework.Fluent/component/Themes/Fluent.Light.xaml" />
    </ResourceDictionary.MergedDictionaries>
  </ResourceDictionary>
</Application.Resources>

xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2008"
xmlns:local="clr-namespace:FluentExample;assembly=FluentExample"
mc:Ignorable="d" ThemeMode="Light"
Title="Fluent스타일" Height="400" Width="600">
  <Grid Style="{x:Null}">
    <Grid.ColumnDefinitions>
      <ColumnDefinition Width="*" Style="{x:Null}" />
      <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <!-- 기본 WPF 스타일 -->
    <StackPanel Grid.Column="0" Margin="20" Style="{x:Null}">
      <TextBlock Text="기본 WPF 스타일" FontWeight="Bold" FontSize="18" Margin="0,0,0,16" Style="{x:Null}" />
      <Button Content="Button" Margin="0,0,0,8" Style="{x:Null}" />
      <TextBox Width="150" Margin="0,0,0,8" Style="{x:Null}" />
      <CheckBox Content="CheckBox" Margin="0,0,0,8" Style="{x:Null}" />
      <ListBox Height="100" Width="150" Style="{x:Null}">
        <ListBoxItem Content="Item 1" Style="{x:Null}" />
        <ListBoxItem Content="Item 2" Style="{x:Null}" />
      </ListBox>
    </StackPanel>
    <!-- Fluent UI 스타일 -->
    <StackPanel Grid.Column="1" Margin="20">
      <TextBlock Text="Fluent UI 적용" FontWeight="Bold" FontSize="18" Margin="0,0,0,16" />
      <Button Style="{DynamicResource AccentButtonStyle}" Content="Button" Margin="0,0,0,8" />
      <TextBox Width="150" Margin="0,0,0,8" />
      <CheckBox Content="CheckBox" Margin="0,0,0,8" />
      <ListBox Height="100" Width="150">
        <ListBoxItem Content="Item 1" />
        <ListBoxItem Content="Item 2" />
      </ListBox>
    </StackPanel>
  </Grid>
</Window>
```

6 .NET10 in WPF

.NET Conf 2025 @Seoul

Fluent UI 스타일 확대

지원 컨트롤 추가

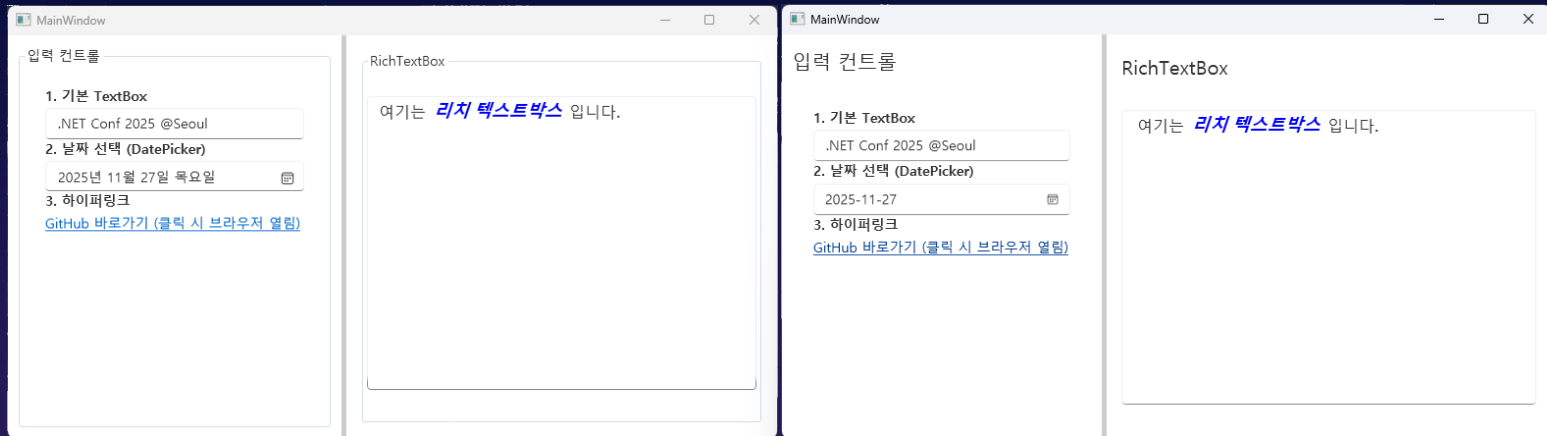
DatePicker, GridSplitter, GroupBox, Hyperlink, Label, RichTextBox, TextBox, NavigationWindow 등 Fluent 스타일 적용

디테일 개선

Expander 컨트롤 애니메이션이 부드러워졌고, 고대비(High Contrast)모드에서의 충돌 버그 수정

RTL(우측에서 좌측쓰기) 지원

MenuItem, TreeViewItem 등에서 레이아웃이 깨지던 문제가 해결됨



참조 0개

```
private void serializePerson()
{
    // 객체를 클립보드에 JSON으로 저장
    var person = new Person { Name = "홍길동", Age = 30 };
    Clipboard.SetDataAsJson("Person", person);

    // 클립보드에서 JSON 데이터로 객체 복원
    if (Clipboard.TryGetData("Person", out Person retrieved))
    {
        Debug.WriteLine($"{retrieved.Name}, {retrieved.Age}");
    }
}
```

참조 4개

```
public class Person
{
    참조 4개
    public string Name { get; set; }
    참조 4개
    public int Age { get; set; }
}
```

클립보드 API 통합 및 보안강화

클립보드란?

컴퓨터에서 텍스트, 이미지, 파일등 다양한 데이터를 임시로 저장하는 공간

클립보드 API 통합

WPF(System.Windows.Clipboard) 및 Windows Forms(System.Windows.Forms.Clipboard)에서 동일한 클립보드 API를 사용

클립보드 보안강화

일부 클립보드에서 사용되는 BinaryFormatter가 더 이상 지원되지 않음, JSON Serialization을 사용하는 API 추가

```
namespace System.Windows
{
    public enum MessageBoxButton
    {
        OK = 0,
        OKCancel = 1,
+       AbortRetryIgnore = 2,
        YesNoCancel = 3,
        YesNo = 4,
+       RetryCancel = 5,
+       CancelTryContinue = 6,
    }

    public enum MessageBoxResult
    {
        None = 0,
        OK = 1,
        Cancel = 2,
+       Abort = 3,
+       Retry = 4,
+       Ignore = 5,
        Yes = 6,
        No = 7,
+       TryAgain = 10,
+       Continue = 11,
    }
}
```

MessageBox API 개선

AbortRetryIgnore

디스크/네트워크/IO 에러 상황에서 중단할지, 계속 시도할지, 무시할지 선택

RetryCancel

일시적인 오류 또는 네트워크 타임아웃 상황에서 다시 시도할지, 그만둘지 선택

CancelTryContinue

긴 처리 과정에서 일부 단계가 실패하는 경우 작업을 취소할지, 방금 단계만 재시도할지, 넘어갈지 선택

6 .NET10 in WPF

.NET Conf 2025 @Seoul

```
<!-- .NET9이하 버전 -->
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*" />
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="2*" />
        <ColumnDefinition Width="3*" />
    </Grid.ColumnDefinitions>
    <!-- ... -->
</Grid>

<!-- .NET10 업데이트 -->
<Grid RowDefinitions="Auto,*,Auto"
ColumnDefinitions="2*,3*">
    <!-- ... -->
</Grid>
```

Grid Xaml 개선

Grid Xaml 개선

Grid.RowDefinitions, Grid.ColumnDefinitions를 간결하게 작성할 수 있도록 개선

WPF는...



지원 및 안정성 확보

.NET 10 장기지원(LTS), 20년간
수 많은 엔터프라이즈 환경에서
안정성과 신뢰성이 검증



성능 및 UI 현대화 지속

.NET10 업데이트에서 확인 했듯이
XAML파싱 속도 향상, 내부 데이터
구조 개선, Fluent UI 제공등 현대화
지속



조용한 강자

B2B, 산업용 개발 도구로 오랫동안
유효할 전망

감사합니다.